
Redyx — Auth Service Test Cases

idityaGE

2026-05-10

Contents

Auth Service — Test Case Specification

Test Infrastructure

1. Hasher Tests (HashPassword / VerifyPassword — Argon2id)

2. JWT Tests (JWTManager.Generate)

3. OTP Tests (OTPManager — Redis-backed)

4. hashRefreshToken Tests

5. Server Input Validation Tests

Patterns & Conventions

2

2

3

3

4

4

6

Auth Service — Test Case Specification

Project Name	Redyx
Module Name	Auth
File	internal/auth/server_test.go
Total Tests	23
Package	auth (same-package test)
Created By	idityaGE
Date of Creation	2026-05-10

Test Infrastructure

Component	Implementation
Logger	zap.NewNop()
Redis	miniredis.RunT(t) (in-process)
DB	nil — validation-only tests short-circuit before DB access
Helper convention	t.Helper() in all setup helpers

1. Hasher Tests (HashPassword / VerifyPassword — Argon2id)

Test Case ID	Test Name	Pre-Condition	Test Steps	Test Data	Expected Result	Status
TC_AUTH_001	TestHashPasswordArgon2id ProducesEncodedHash	Argon2id Default-Params available	Call HashPassword("mysecretpassword", DefaultParams)	password = "mysecretpassword"	Returns a non-empty encoded hash string; no error	PASS
TC_AUTH_002	TestHashPasswordArgon2id DifferentSaltEachTime	Argon2id Default-Params available	Call HashPassword twice with same password; compare hashes	password = "same-password"	Two hashes are different (unique random salt per call)	PASS
TC_AUTH_003	TestVerifyPasswordValidHash CorrectPassword	Valid hash produced by HashPassword	Hash password; call VerifyPassword with same password	password = "correctorsebatterystaple"	Returns valid=true, no error	PASS

Test Case ID	Test Name	Pre-Condition	Test Steps	Test Data	Expected Result	Status
TC_AUTH_004	TestVerifyPassWrongPassword	Valid hash produced by HashPassword	Hash "correct-password"; verify with "wrong-password"	correct = "correct-password", wrong = "wrong-password"	Returns valid=false, no error	PASS
TC_AUTH_005	TestVerifyPassword_InvalidHash-Format		Call VerifyPassword("password", valid-hash "not-a-valid-hash")	hash = "not-a-valid-hash"	Returns an error (invalid format)	PASS

2. JWT Tests (JWTManager.Generate)

Test Case ID	Test Name	Pre-Condition	Test Steps	Test Data	Expected Result	Status
TC_AUTH_006	TestJWTManager_JWTManagerGenerate_ReturnsToken	JWTManager created with "test-secret-key" and 15 min TTL	Call Generate("user-123", "testuser")	userID = "user-123", username = "testuser"	Returns non-empty token string; expiresAt is in the future	PASS
TC_AUTH_007	TestJWTManager_JWTManagerGenerate_ExpiredRespected	JWTManager with 30 min TTL	Generate token; compare expiresAt with now + 30m	TTL = 30 min	expiresAt within ±5 s of now + 30m	PASS
TC_AUTH_008	TestJWTManager_JWTManagerGenerate_DifferentUsers_DifferentTokens	JWTManager with 15 min TTL	Generate tokens for two users; compare	user-1/alice vs user-2/bob	Tokens are different	PASS

3. OTP Tests (OTPManager — Redis-backed)

Test Case ID	Test Name	Pre-Condition	Test Steps	Test Data	Expected Result	Status
TC_AUTH_009	TestOTPManager_miniredisGenerate_StoresOTP	miniredis running, 5 min TTL	Call Generate(ctx, "user@example.com")	email = "user@example.com"	Returns exactly a 6 digit code string; no error	PASS

Test Case ID	Test Name	Pre-Condition	Test Steps	Test Data	Expected Result	Status
TC_AUTH_010	TestOTPManager_Verify_CorrectCode	OTP generated for email	Generate code; call Verify with same code	email = "user@example.com"	Returns valid=true, no error	PASS
TC_AUTH_011	TestOTPManager_Verify_WrongCode	OTP generated for email	Generate code; verify with "000000"	wrong code = "000000"	Returns valid=false, no error	PASS
TC_AUTH_012	TestOTPManager_Verify_SingleUse	OTP generated for email	Verify correct code twice in sequence	same code used twice	1st verify → valid=true; 2nd verify → valid=false (OTP consumed)	PASS
TC_AUTH_013	TestOTPManager_Verify_NonExistentCode	No OTP generated for email	Call Verify for unknown email with "123456"	email = "no-body@example.com"	Returns valid=false, no error	PASS

4. hashRefreshToken Tests

Test Case ID	Test Name	Pre-Condition	Test Steps	Test Data	Expected Result	Status
TC_AUTH_014	TestHashRefreshToken_Deterministic		Call hashRefreshToken twice with same input; compare	token = "some-token-value"	Both hashes are identical (deterministic SHA-256)	PASS
TC_AUTH_015	TestHashRefreshToken_DifferentInputs_DifferentHashes		Call hashRefreshToken with two distinct tokens; compare	"token-a" vs "token-b"	Hashes are different	PASS
TC_AUTH_016	TestHashRefreshToken_NonEmpty		Call hashRefreshToken("any token")	token = "any token"	Returns a non-empty string	PASS

5. Server Input Validation Tests

5.1 Register

Test Case ID	Test Name	Pre-Condition	Test Steps	Test Data	Expected Result	Status
TC_AUTH_017	TestRegister_InvalidEmail	Auth server initialized (nil DB)	Call Register with 4 invalid email sub-cases	"" , "notanemail", "user@", "user@domain"	All sub-cases return an error; no nil error	PASS
TC_AUTH_018	TestRegister_InvalidUsername	Auth server initialized (nil DB)	Call Register with 4 invalid username sub-cases	"ab" (too short), "averylongusername that exceeds limit" (too long), "user name" (space), "user-name" (dash)	All sub-cases return an error	PASS
TC_AUTH_019	TestRegister_ShortPassword	Auth server initialized (nil DB)	Call Register with password < 8 chars	password = "short"	Returns an error	PASS

5.2 Login

Test Case ID	Test Name	Pre-Condition	Test Steps	Test Data	Expected Result	Status
TC_AUTH_020	TestLogin_EmptyEmail	Auth server initialized	Call Login with empty email	email = ""	Returns an error	PASS
TC_AUTH_021	TestLogin_EmptyPassword	Auth server initialized	Call Login with empty password	password = ""	Returns an error	PASS

5.3 VerifyOTP / RefreshToken / Logout / ResetPassword / GoogleOAuth

Test Case ID	Test Name	Pre-Condition	Test Steps	Test Data	Expected Result	Status
TC_AUTH_022	TestVerifyOTP_EmptyEmail	Auth server initialized	Call VerifyOTP with empty email	email = "", code = "123456"	Returns an error	PASS
TC_AUTH_023	TestVerifyOTP_EmptyCode	Auth server initialized	Call VerifyOTP with empty code	email = "user@example.com", code = ""	Returns an error	PASS
TC_AUTH_024	TestRefreshToken_EmptyToken	Auth server initialized	Call RefreshToken with empty refresh token	refreshToken = ""	Returns an error	PASS

Test Case ID	Test Name	Pre-Condition	Test Steps	Test Data	Expected Result	Status
TC_AUTH_025	TestLogout_EmptyToken_ReturnsSuccess	Auth server initialized	Call Logout with empty refresh token	refreshToken = ""	Returns success (idempotent — logout with missing token is a no-op)	PASS
TC_AUTH_026	TestResetPassword_NoFieldsProvided	Auth server initialized	Call ResetPassword with empty email	email = ""	Returns an error	PASS
TC_AUTH_027	TestResetPassword_OnlyTokenProvided	Auth server initialized	Call ResetPassword with token but no new_password	token = "some-token", newPassword = ""	Returns an error	PASS
TC_AUTH_028	TestGoogleOAuth_NoOAuthConfigured	Auth server with nil OAuth config	Call GoogleOAuth with a non-empty code	code = "some-code"	Returns an error (OAuth not configured)	PASS
TC_AUTH_029	TestGoogleOAuth_EmptyCode	Auth server initialized	Call GoogleOAuth with empty code	code = ""	Returns an error	PASS

Patterns & Conventions

Pattern	Implementation
Package scope	package auth (white-box / same-package)
Logger	zap.NewNop() (silent)
In-process Redis	miniredis.RunT(t) — auto-cleaned on t.Cleanup
Helper tagging	t.Helper() in testRedisClient, testAuthServer
Nil DB tolerance	Server constructed with nil DB pool; tests short-circuit at validation layer
Auth context	auth.WithClaims(context.Background(), &auth.Claims{...}) for authenticated RPCs